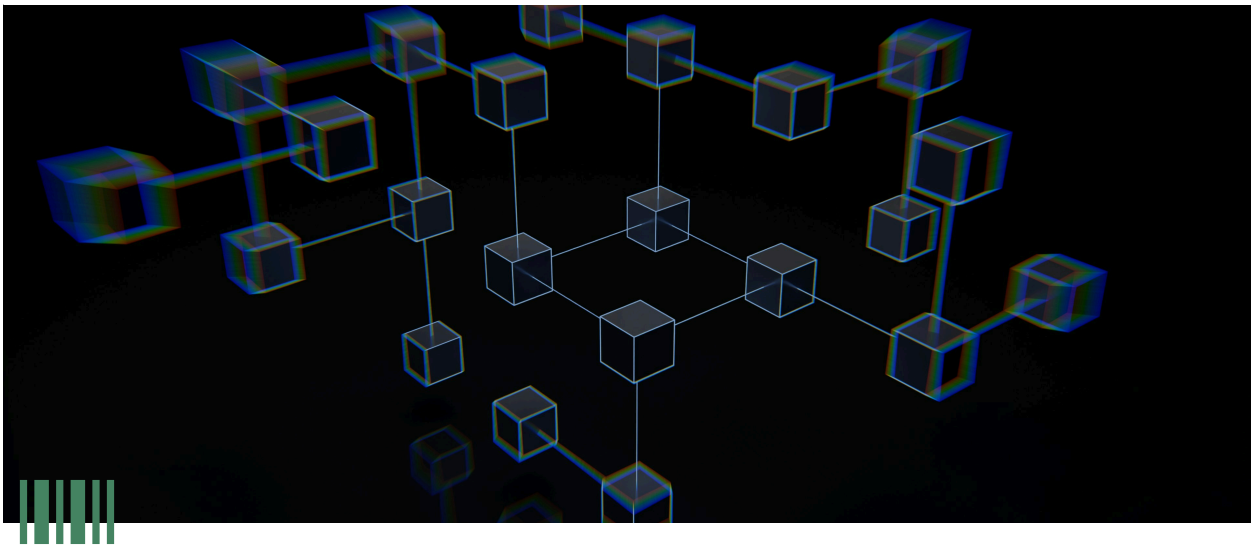


Methodology

Created	November 13, 2025 12:27 AM
Tags	

Owner: ***Mohamed Hamdi***

Full Recon Methodology: Subdomain Enumeration, Infrastructure & URL Discovery

This document provides a complete breakdown of all commands, tools, OSINT services, DNS wordlists, and filtering methodology for automated target reconnaissance.

STEP 1: Subdomain Enumeration (Passive & Active)

Passive Subdomain Enumeration

subfinder

- `subfinder -d example.com -o Subs01.txt` *# Runs Subfinder passively on target domain and outputs results to Subs01.txt.*
- `subfinder -dL example.com -o Subs01.txt` *# Passes a domain list (or target file) to Subfinder for passive discovery.*
- `subfinder -d example.com -all -recursive -o Subs01.txt` *# Uses all passive sources and enables recursive enumeration for nested subdomains.*

Assetfinder

- `echo example.com | assetfinder --subs-only > Subs02.txt` *# Pipes the target domain into Assetfinder to discover related subdomains, saving subdomains only.*

---

amass

- `amass enum -d example.com -o Subs03.txt` *# Runs Amass active/passive OSINT enumeration to gather subdomains into Subs03.txt.*
- `amass enum -d example.com -brute -w wordlist.txt -o Subs03.txt` *# Enables DNS bruteforcing in Amass using a specified custom wordlist.*

---

subenum

- `./subenum.sh -d example.com -u wayback,crt,abuseipdb,Findomain,Subfinder,Amass,Assetfinder -o Subs04.txt` *# Runs subenum script against a single target domain across multiple enumerated API services.*
- `./subenum.sh -l targets.txt -u wayback,crt,abuseipdb,Findomain,Subfinder,Amass,Assetfinder -o Subs04.txt` *# Runs subenum script against a list of target domains specified in targets.txt.*

---

findomain

- `findomain -t example.com -u Subs05.txt` *# Executes Findomain tool for fast cross-platform passive subdomain enumeration.*

---

chaos

- `export PDCP_API_KEY=your_api_key_here` *# Sets the ProjectDiscovery Chaos API key environment variable for access.*
- `chaos -d example.com -o Subs06.txt` *# Queries ProjectDiscovery's dataset for pre-computed subdomains of the target domain.*

---

github-subdomains

- `github-subdomains -d example.com -t YOUR_GITHUB_TOKEN` *# Searches public GitHub repositories for hidden subdomains linked to the target.*
- Reference: `https://github.com/settings/tokens` *# URL to generate a Personal Access Token required for GitHub API requests.*

Active Subdomain Enumeration / Bruteforce

dnscan

- `python dnscan.py -d example.com -w wordlist.txt -t 300 | tee /home/kali/Subs07.txt` *# Performs fast multi-threaded DNS bruteforcing with 300 threads and saves output via tee.*

---

puredns

- `puredns bruteforce wordlist.txt example.com -r resolvers.txt` *# Resolves bruteforced subdomains accurately using custom DNS resolvers.*
- `sudo wget <https://raw.githubusercontent.com/trickest/resolvers/main/resolvers.txt` *># Downloads a fresh, reliable list of public DNS resolvers for puredns.*

Infrastructure Discovery

- `https://bgp.he.net` *# Hurricane Electric BGP Toolkit used to inspect ASN, IP ranges, and routing data.*

asnmap

- `asnmap -a AS17012` *# Maps and retrieves all CIDR IP prefixes associated with a specific ASN.*

whois

- `whois -h whois.radb.net AS17012` *# Queries RADB WHOIS server for detailed Autonomous System registration info.*

dnsx PTR

- `echo 66.211.170.0/23 | dnsx -silent -resp-only -ptr` *# Performs reverse DNS lookup (PTR record query) on an entire CIDR IP block.*

---

hakrevdns

- `hakrevdns -d example.com -R resolvers.txt` *# Performs reverse DNS lookup on IP addresses linked to target domains.*

---

magicrecon

- `./magicrecon.sh -l targets.txt --all` *# Runs an automated multi-tool reconnaissance pipeline against a list of targets.*

Certificate Transparency Subdomains (SSL)

- `curl -s "<https://crt.sh/?q=%25.example.com&output=json>" | jq -r '.[].name_value' | sed 's/\*//g' | tr '\n' ',' | tr ',' '\n' | grep -oE "([a-zA-Z0-9._-]+)\.example\.com" | sort -u` *# Queries crt.sh API, extracts SSL certificate subject names, cleans wildcards, and uniquely filters domain matches.*

OSINT Sources (subdomains + info)

- `https://securitytrails.com` + `https://www.ultimatedomains.com/extract-domains.php` *# Query domain historical records and extract clean domain*

lists.

- `https://shrewdeye.app` *# Automated OSINT target surface and subdomain discovery portal.*
- `https://subdomainfinder.c99.nl` *# Online tool for finding subdomains from public search engine archives.*
- `https://netlas.io` *# Internet-connected device search engine for asset discovery.*
- `https://urlscan.io` *# Analyzes websites, HTTP requests, DOM structures, and linked subdomains.*
- `https://otx.alienvault.com` *# Threat intelligence platform containing historical domain indicator logs.*
- `https://search.censys.io` *# Attack surface search engine for public certificates and hosts.*
- `https://www.virustotal.com` *# Malware & DNS intelligence lookup for broad asset relations.*
- `https://www.favihash.com` *# Discovers assets sharing identical website favicon hashes.*
- `https://crt.sh` *# Search engine for Certificate Transparency logs.*
- `https://bgp.he.net` *# BGP and IP range discovery tool.*
- `https://www.shodan.io + asn:AS8300` *# Discovers devices and services hosted under a specific ASN.*
- `https://search.dnslytics.com/cidr` *# Reverses CIDR blocks to list associated domains and infrastructure.*
- `https://viewdns.info/reverseip/?host=` *# Reverse IP lookup to uncover co-hosted websites on the same IP.*

DNS Wordlists

- `/usr/share/wordlists/seclists/Discovery/DNS/bitquark-subdomains-top100000.txt` *# Top 100,000 subdomains list from SecLists for DNS enumeration.*

- `/usr/share/wordlists/seclists/Discovery/DNS/dns-Jhaddix.txt` *# High-yield target subdomain list compiled by Jason Haddix.*
- `/usr/share/wordlists/seclists/Discovery/DNS/subdomains-top1million-110000.txt` *# Comprehensive 110,000 entry wordlist from SecLists for large-scale scanning.*
- `/usr/share/wordlists/commonspeak2-wordlists-master/subdomains/subdomains.txt` *# BigQuery dataset-generated wordlist reflecting real-world subdomain names.*
- `/usr/share/wordlists/commonspeak2-wordlists-master/subdomains/fierce-hostlist.txt` *# Legacy host list used commonly for initial DNS brute-forcing.*
- `/usr/share/wordlists/best-dns-wordlist.txt` *# Curated master list optimized for high success rate DNS resolving.*
- `sudo wget <https://wordlists-cdn.assetnote.io/data/manual/best-dns-wordlist.txt` *># Command to download Assetnote's regularly updated DNS wordlist.*

STEP 2: Merge & Remove Duplicates

anew

- `cat Subs*.txt | anew | tee AllSubs.txt` *# Combines all generated subdomain text files, filters unique entries using anew, and saves to AllSubs.txt.*
- `wc AllSubs.txt` *# Counts line, word, and byte count to check total unique subdomains found.*

STEP 3: Alive Host Detection

httpx

- `cat AllSubs.txt | httpx -o AliveSubs.txt` *# Probes unique subdomains to check for live HTTP/HTTPS servers and saves active ones to*

AliveSubs.txt.

- `cat AllSubs.txt | httpx -status-code -content-length -web-server -title -follow-redirects` *# Probes targets and outputs HTTP response status, page size, server header, title, and handles redirects.*
- `cat AllSubs.txt | httpx -status-code -content-length -web-server -title -follow-redirects -match-code 200` *# Filters live hosts that explicitly return an HTTP 200 OK status code.*
- `cat AllSubs.txt | httpx -status-code -content-length -web-server -title -follow-redirects -filter-code 400` *# Excludes target responses returning HTTP 400 Bad Request status codes.*

Status Notes & Next Actions

- `404 -> wayback + fuzz` *# For 404 Not Found endpoints, check historical URLs in Wayback machine or perform directory fuzzing.*
- `403 -> try bypass` *# For 403 Forbidden endpoints, attempt access control bypass techniques (headers, alternative paths, IP spoofing).*

URL Discovery and End Point

URL Discovery

- **waybackurls**: [https://github.com/tomnomnom/waybackurls] - A tool to fetch known URLs from the Wayback Machine for a domain.
  `cat AliveSubs.txt | waybackurls > WB1.txt`
- **waymore**: [https://github.com/xnl-h4ck3r/waymore] - Advanced tool to fetch archived URLs and response bodies from multiple

sources like Wayback and URLScan.

```
waymore -i AliveSubs.txt -mode U -l 1000 -from 2021 -oU WM1.txt
```

- gau (GetAllUrls): [https://github.com/lc/gau] - Fetches known URLs from AlienVault's OTX, Wayback Machine, and Common Crawl.

```
cat AliveSubs.txt | gau --threads 200 > GAU1.txt
```

- gauplus: [https://github.com/bp0lr/gauplus] - An optimized and faster version of gau for gathering archived URLs concurrently.

```
gauplus -t 200 -random-agent < AliveSubs.txt > GAU2.txt
```

- hakrawler: [https://github.com/hakluke/hakrawler] - Fast and lightweight web crawler for discovering endpoints and assets on a target.

```
cat AliveSubs.txt | hakrawler -subs -u -insecure > HK1.txt
```

- katana: [https://github.com/projectdiscovery/katana] - Next-generation crawling framework supporting headless browsing for JS-heavy web apps.

```
katana -u AliveSubs.txt -jc -kf all -d 5 -headless -fx -aff -fs rdn -f url -silent > KTN1.txt
echo " example.com " | katana
```

- gospider: [https://github.com/jaeles-project/gospider] - Fast web spider written in Go that extracts links, sitemaps, robots.txt, and JS endpoints.[cite: 2]

```
gospider -S AliveSubs.txt -t 20 -d 3 --js --sitemap --robots -o GS1.txt[cite: 2]
gospider -S httpx.txt | sed -n 's/. \(https:\/\/[^ ]\). /\1/p' >> GS2.txt[cite: 2]
```

- paramspider: [https://github.com/devanshbatham/paramspider] - Parameter discovery tool designed to find URLs with parameters for testing vulnerabilities (XSS, SQLi, etc.).[cite: 2]

```
paramspider -d example.com -o PS1.txt[cite: 2]
```

- x8: [https://github.com/ShreshthMohan/x8] - Hidden parameter discovery suite written in Rust for fast fuzzing and parameter discovery.[cite: 2]

```
x8 -u "AliveSubs.txt" -o X81.txt[cite: 2]
```

- -Merge URLs: cat WB1.txt GAU1.txt GAU2.txt KTN1.txt HK1.txt GS1.txt GS2.txt PS1.txt X81.txt | anew | tee AllURLs.txt[cite: 2]

Extract Interesting Data:

- `cat AllURLs.txt | grep[cite: 2]`

```
        grep -i "\.js" > js.txt[cite: 2]
    grep -i "\.asp" > asp.txt[cite: 2]
    grep -i "\.php" > php.txt[cite: 2]
    grep -i "\.jsp" > jsp.txt[cite: 2]
    grep -i "\.jspx" > jspx.txt[cite: 2]
    grep -i "\.aspx" > aspx.txt[cite: 2]
    grep "=" > injection.txt[cite: 2]
```

- Discover Interesting Points:
  - urinteresting: [https://github.com/m4ll0k/urinteresting] - A tool designed to filter and extract interesting or potentially vulnerable URLs from a list based on regex patterns.
    `cat AllURLs.txt | urinteresting`

Extract Parameters and Endpoints

- A comprehensive guide for extracting parameters, API endpoints, sensitive files, and specific attack vectors from target URLs using advanced Regex pattern matching and CLI tools.
  - Parameter Extraction

```
#qsreplace: [https://github.com/tomnomnom/qsreplace]
Replaces all query parameter values in URLs with a cust
om string ( `FUZZ`) for vulnerability testing (XSS, SQL
i, SSRF).
```

```bash
cat AllURLs.txt | grep "=" | qsreplace "FUZZ" | anew |
tee ParamURLs.txt
```

```
#anew: [https://github.com/tomnomnom/anew]
Filters out duplicate URLs containing parameter query d
elimiters (`=`) and appends only unique lines.
```bash
cat AllURLs.txt | grep "=" | anew params.txt
```
```

- Parameter Key Stripping

```
#Extracts clean URLs with empty parameter values (strip
s existing parameter values after `=`) to prepare clean
templates for automated scanners.
```bash
cat AllURLs.txt | grep '=' | sed -E 's/=[^&]*/=/g' | an
ew param.txt
```
```

- JavaScript Discovery

```
# JS Files Filtering (Advanced Regex)
# Matches JavaScript files accurately, considering quer
y parameters, cache-busting strings, and ignoring stati
c `.json` files.
```bash
cat AllURLs.txt | grep -Ei "\.js(\?.*)?$" | anew js_url
s.txt
cat AllURLs.txt | grep -iE "\.jsx?(\?.*)?$" | grep -ivE
"\.json" | sort -u | tee js.txt
```
Wayback Machine CDX API Query
```

```
# Directly queries the Wayback Machine's CDX index to r
etrieve all historically archived JavaScript endpoints
for a target domain.
```bash
curl -s "https://web.archive.org/cdx/search/cdx?url=*.e
xample.com/*&collapse=urlkey&output=text&fl=original&fi
lter=original:.*\.js$" | anew js_wayback.txt
```
```

- Advanced Endpoint & Parameter Categorization (Enhanced Regex)

```
# API Endpoints Discover
# Extracts endpoints returning structured data formats
(JSON, XML, GraphQL, REST APIs, and versioned API path
s).
```bash
cat AllURLs.txt | grep -Ei "(\/api\/v[0-9]|\.(json|xml|
graphql|gql|swagger|openapi))(\?.*)?$" | anew api_urls.
txt
```
Backend File Extensions
Identifies server-side script files and dynamic web a
pplication pages.
```bash
cat AllURLs.txt | grep -Ei "\.(php|asp|aspx|jsp|jspx|cf
m|cgi|pl|rb|py)(\?.*)?$" | anew backend_files.txt
```
Authentication & Login Flows
Extracts URLs associated with user authentication, OA
uth flows, registration, and password recovery mechanis
ms.
```bash
cat AllURLs.txt | grep -Ei "(login|signin|signup|regist
er|auth|oauth|sso|token|reset|password|account|sessio
```
```

```
n)" | anew login_flows.txt
```
File Operations & Upload Endpoints
Targets file management, upload forms, downloads, and media assets that could be vulnerable to File Upload / LFI bugs.
```bash
cat AllURLs.txt | grep -Ei "(upload|file|download|image|media|avatar|import|export|attachment|doc)" | anew file_uploads.txt
```
Administrative Panels
Filters administrative, internal management interfaces, and restricted backend boards.
```bash
cat AllURLs.txt | grep -Ei "(admin|dashboard|internal|manage|manager|root|controlpanel|cpanel|portal|sysadmin)" | anew admin_panels.txt
```
Sensitive Files & Configuration Leaks
Discovers exposed environment files, backups, configuration settings, databases, and server logs.
```bash
cat AllURLs.txt | grep -Ei "\.(env|bak|config|conf|sql|log|swp|git|yaml|yml|json|db|tar|gz|zip)(\?.*)?$" | anew sensitive_files.txt
```
IDOR & Numeric Parameter Targets
Extracts endpoints containing numeric IDs, UUIDs, or GUIDs in path or query strings for IDOR testing.
```bash
cat AllURLs.txt | grep -Ei "(\/|\=)[0-9]{1,10}(\/|\?|&|$)| [0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12} " | anew idor_targets.txt
```
High-Value & Open Redirect / SSRF Targets

```
# Targets parameters frequently susceptible to Open Redirect, SSRF, RCE, or Sensitive Data Access.
```bash
cat AllURLs.txt | grep -Ei "(admin|login|signup|redirect|callback|auth|dev|test|beta|debug|staging|url=|r=|u=|goto=|return=|dest=|next=|target=|path=|file=|page=|fetch=|proxy=)" | anew interesting.txt
```
Cloud Secrets, Tokens & API Keys
Filters URLs and paths containing exposed API keys, cloud credentials, tokens, and storage buckets.
```bash
cat AllURLs.txt | grep -Ei "(aws|s3|bucket|gcp|azure|vault|token|apikey|secret|key|bearer|access_token|private)" | anew cloud_leaks.txt
```
```

JavaScript Secret Discovery

- mantra: [https://github.com/MrSudoX/mantra]
  - A fast tool for discovering sensitive information, API keys, and endpoints leaked inside JavaScript files. `cat js_urls.txt | mantra`
- jsecret: [https://github.com/Rsthn/jsecret]
  - A script designed to scan JavaScript source code files for hardcoded secrets, API tokens, and confidential endpoints. `cat js_urls.txt | jsecret`
- jsleak: [https://github.com/chaitin/jsleak]
  - A high-performance concurrent tool for discovering secrets, endpoints, and links inside JS files withmulti-threading support. `cat js_urls.txt | xargs -P 20 -I {} jsleak -s -l -k -e {} >> jsleak_output.txt`
- Manual Pattern Matching (Regex Secrets)

- Searches through downloaded JavaScript source code files using Regular Expressions to identify common parameter patterns like API keys, tokens, secrets, and passwords.

  `grep -E "api[_-]?key|token|secret|password" file.js`

Direct JavaScript File Analysis

- Parses a specific JS file to extract embedded endpoints, API paths, or secret parameters.

```
# Lazyegg Methodology & Usage Guide

## Overview

**Lazyegg** is an automated Python reconnaissance scrip
t designed to parse, extract, and discover hidden endpo
ints, subdomains, IP addresses, and sensitive parameter
s from web applications and JavaScript files.

---

## Tools Included

### 1. Lazyegg
* **Repository:** [https://github.com/m4ll0k/Lazyegg](h
ttps://github.com/m4ll0k/Lazyegg)
* **Description:** A Python-based tool used to scan tar
gets or specific JavaScript files for endpoints, parame
ters, hidden URLs, and embedded domains/IPs.
$ python lazyegg.py http://example.com/js/auth.js
### 2. waybackurls
* **Repository:** [https://github.com/tomnomnom/wayback
urls](https://github.com/tomnomnom/waybackurls)
* **Description:** A command-line utility by TomNomNom
to fetch all known URLs that the Wayback Machine has ca
ched for a given target domain.
```

```
$ waybackurls vulnweb.com | grep '\.js$' | awk -F '?'
'{print $1}' | sort -u | xargs -I{} bash -c 'python laz
yegg.py "{}" --js_urls --domains --ips' > jsurls.log &&
cat jsurls.log | grep '\.' | sort -u
---

## Commands & Usage Examples

### 1. Single Target Reconnaissance
Scans a target domain directly for URLs, endpoints, and
parameters.
```bash
python lazyegg.py [http://example.com](http://example.c
om)
```

Find API Keys, Hidden Parameters & Open Ports

- Secret Discovery
  - TruffleHog: [https://github.com/trufflesecurity/trufflehog] A high-performance scanner designed to find secrets, API keys, credentials, and tokens across GitHub organizations, filesystems, and Git repositories.

```
Scan GitHub Organization Secrets:
  ```bash
  trufflehog github --org=example-org --results=verifie
d
  ```
Scan Filesystem / JS Files:
  ```bash
  trufflehog filesystem js_urls.txt --json > trufflehog
_results.json
  ```
```
```

Scan Git Repository:
  ```bash
  trufflehog git https://github.co

- PHP Parameter Discovery & SQL Injection
  - Arjun: [https://github.com/s0md3v/Arjun]
    An automated parameter discovery tool used to find hidden HTTP query parameters in web applications.

    ```bash
    arjun -i php_urls.txt >> php_parameters.txt
    ```
    ```bash
    sqlmap -u "https://example.com/file.php?id=*" --dbs --b
    anner --batch --random-agent
    ```

- Full Vulnerability Scan
  - Nuclei: [https://github.com/projectdiscovery/nuclei]
    A fast, template-based vulnerability scanner used to send requests across targets based on community-curated vulnerability templates.

    Scan Targets List for Exposure Vulnerabilities:
      ```bash
      nuclei -l all.txt -t /nuclei-templates/http/exposures
    -o results.txt
      ```
    Scan Single URL for Exposure Vulnerabilities:
      ```bash
      nuclei -u https://www.example.com -t /nuclei-template
    s/http/exposures
      ```

- Hidden Parameters Discovery
      ```

    - Param Miner (Burp Suite Extension):
      [https://github.com/PortSwigger/param-miner]
      A Burp Suite extension designed to identify hidden, unlinked parameters (useful for finding Web Cache Poisoning, SSRF, and key-handling bugs).
      * **Usage:** `paraminer` (Run inside Burp Suite interface).

````
### Arjun (Parameter Brute-Forcing)
* **Scan Targets List to Output JSON:**
  ```bash
  arjun -i AllURLs.txt -o arjun.json
  ```

* **Scan Single URL (GET Request):**
  ```bash
  arjun -u https://example.com/test
  ```

* **Scan Single URL (POST Request):**
  ```bash
  arjun -u https://example.com/test -m post
  ```

### ffuf (Fuzzing Parameters): [https://github.com/ffuf/ffuf]
A fast web fuzzing tool written in Go, used here to brute-force query parameter names.
```bash
ffuf -u http://example.com/search?FUZZ=value -w wordlist.txt
````

- Port Discovery

````
### Naabu: [https://github.com/projectdiscovery/naabu]
A fast port scanner written in Go focused on reliability a
````
````

````
nd simplicity.
```bash
naabu -list AliveSubs.txt -p - -rate 2000 -o ports.txt
```

### Nmap: [https://github.com/nmap/nmap]
The industry-standard network mapper and security scanner used for network discovery and vulnerability auditing.
```bash
nmap -iL AliveSubs.txt -T4 -Pn
````

Sensitive File Enumeration

- Advanced Wayback Sensitive File Enumeration
  - waybackurls: [https://github.com/tomnomnom/waybackurls] Fetches historical URLs from the Wayback Machine and filters them using an extensive regex pattern to identify sensitive files (spreadsheets, SQL dumps, backups, archives, config files, `.htaccess`, and `.env` files).

````
``bash
waybackurls https://www.hackerone.com | grep --color=alway
s -E "\.xls|\.xlsx|\.csv|\.sql|\.db|\.bak|\.backup|\.old|
\.tar\.gz|\.tgz|\.zip|\.7z|\.rar|\.pdf|\.doc|\.docx|\.pptx
|\.txt|\.log|\.ini|\.conf|\.config|\.yarnrc|\.bowerrc|\.es
lintrc|\.eslintignore|\.prettierrc|\.prettierignore|\.styl
elintrc|\.stylelintignore|\.htgroup|\.htusers|\.htdigest|
\.htdbm|\.htpasswds|\.htaccess\.bak|\.htaccess\.old|\.htac
cess\.backup|\.ht"  ```
````

- Sensitive Data Leaks Using Google Sheets
  - Advanced Google Dorking techniques to discover exposed Google Spreadsheets containing passwords, emails, or internal target data.
````

```text
site:*.target.com intext:"docs.google.com/spreadsheets"
site:docs.google.com/spreadsheets "target.com"
site:docs.google.com/spreadsheets "password" "test.com"
site:docs.google.com/spreadsheets "@test.com"
```

- GitHub Dorks & OSINT Search Tools
  - GitDorker: [https://github.com/obheda12/GitDorker]
    A python tool that utilizes GitHub Search API and dorks to discover sensitive data and leaked secrets across GitHub repositories.

```bash
python3 GitDorker.py -tf tf/TOKENSFILE -q example.com -
d dorks/medium_dorks.txt -o example
```

Useful GitHub Dork Resources & Links:
* **GitHub Dorks Collection:** [https://github.com/Prov
iesec/github-dorks](https://github.com/Proviesec/github
-dorks)
* **Interactive GitHub Dorks Helper:** [https://mr-koan
ti.github.io/github.html](https://mr-koanti.github.io/g
ithub.html)

- Backup File Artifacts Checker
  - BFAC (Backup File Artifacts Checker):
    [https://github.com/mazen160/bfac]
    An automated tool designed to discover backup files, source code leaks, and residual development files on target web servers.

```bash
bfac --url https://example.com --detection-technique al
l --level 3 --exclude-status-codes 404,500
```

Fuzzing Directory

- Directory Fuzzing & Brute-Forcing
  - dirb: [https://github.com/vspair/DIRB]
    A classic Web Content Scanner that looks for existing/hidden Web Objects via dictionary attack
  - feroxbuster: [https://github.com/epi052/feroxbuster]
    A fast, simple, and recursive content discovery tool written in Rust.
  - gobuster: [https://github.com/OJ/gobuster]
    A high-performance directory/file, DNS, and VHost brute-forcing tool written in Go.

    ```bash
    dirb https://example.com
    ```

    ```bash
    feroxbuster -u https://example.com -w wordlist.txt -t 3
    00 -k -d 3 -e -x php,html,json,js,log,txt,bak,old,zip,t
    ar,gz
    ```

    ```bash
    gobuster dir -u https://example.com -w wordlist.txt -k
    ```

- Directory Scanning (dirsearch)

- dirsearch: [https://github.com/maurosoria/dirsearch]
  An advanced command-line tool designed to brute force directories and files in webservers.

````
* **Filter Status Code 200 & Output to File:**
  ```bash
  python3 dirsearch.py -u https://example.com -i 200 --full-url -o result.txt
  ```

* **Recursive Directory Scanning:**
  ```bash
  python3 dirsearch.py -u https://example.com -i 200 --recursive
  ```

* **Deep Recursive Scan with Random User-Agent:**
  ```bash
  python3 dirsearch.py -u https://example.com --random-agent --full-url --deep-recursive
  ```

* **HTTP POST Method Scanning:**
  ```bash
  python3 dirsearch.py -u https://example.com -m post
  ```

* **Custom Extensions Enumeration:**
  ```bash
  python3 dirsearch.py -u https://example.com --extensions=php,asp,aspx,jsp,json,xml,txt,log,ini,cfg,config,conf,bak,old,backup,zip,tar,tgz,gz,rar,7z,swp,swo,db,sql
  ```

* **Include / Exclude Specific Status Codes:**
  ```bash
````
````

```
  python3 dirsearch.py -u https://example.com --include
-status=200,204,301,302,307,308,401,403 --exclude-statu
s=404
  ```

* **High-Speed Multi-Threaded Scan with Wordlist & Exte
nsions:**
  ```bash
  python3 dirsearch.py -u https://example.com -w wordli
st.txt -e php,asp,aspx,jsp,json,xml,txt,log,ini,cfg,con
fig,conf,bak,old,backup,zip,tar,tgz,gz,rar,7z,swp,swo -
t 80 -r --full-url
  ```

* **Recursive Depth Scan with Selected Extensions:**
  ```bash
  python3 dirsearch.py -u https://example.com -r -R 3 -
e php,html,js,json,txt,bak,zip,tar,gz,old,backup,conf
```

- Advanced FFUF Techniques
  - ffuf: [https://github.com/ffuf/ffuf]
    A fast web fuzzing tool written in Go.

```
* **High-Speed Directory Fuzzing (Status Code 200):**
  ```bash
  ffuf -u https://example.com/FUZZ -w wordlist.txt -t 2
000 -mc 200
  ```

* **Custom Port & HTTP Method (POST):**
  ```bash
  ffuf -u https://example.com:80/FUZZ -w wordlist.txt -
X POST
  ```
```

````
* **Direct IP Address Target Fuzzing:**
  ```bash
  ffuf -u https://192.1.2.3:80/FUZZ -w wordlist.txt
  ```

* **Multi-Wordlist Fuzzing (Directory + File/Paramete
r):**
  ```bash
  ffuf -u https://example.com/FUZZ/FUZZX -w list1.txt:F
UZZ -w list2.txt:FUZZX
  ```

* **Bypass Rate Limiting via Custom Header (`X-Forwarde
d-For`):**
  ```bash
  ffuf -u https://example.com/FUZZ -w wordlist.txt -H
"X-Forwarded-For: 127.0.0.1" -H "X-Forwarded-Host: 127.
0.0.1"
  ```

* **Fuzzing with File Extensions:**
  ```bash
  ffuf -u http://example.com/FUZZ -w wordlist.txt -e .h
tml,.php,.asp,.aspx,.js,.json,.xml,.config,.bak,.old,.b
ackup,.zip,.rar
  ```

* **Authenticated Fuzzing (Bearer Token):**
  ```bash
  ffuf -u http://example.com/FUZZ -w wordlist.txt -H "A
uthorization: Bearer TOKEN"
  ```

* **Recursive Fuzzing:**
  ```bash
  ffuf -u http://example.com/FUZZ -w wordlist.txt -recu
````
````

````
rsion
  ```

* **Recursive Fuzzing with Depth Limit:**
  ```bash
  ffuf -u http://example.com/FUZZ -w wordlist.txt -recu
rsion-depth 2
  ```

* **Route Traffic Through Proxy (Burp Suite / OWASP ZA
P):**
  ```bash
  ffuf -u http://example.com/FUZZ -w wordlist.txt -x ht
tp://127.0.0.1:8080
  ```

* **Credential Fuzzing (Username & Password Brute-Forc
e):**
  ```bash
  ffuf -u https://example.com/?username=FUZZ&password=F
UZZX -w username.txt:FUZZ -w password.txt:FUZZX
  ```

* **Raw HTTP Request File Fuzzing:**
  ```bash
  ffuf -request fuzzing_login.txt -request-proto http -
w password.txt:PASS
  ```

* **Parameter & Directory Matrix Fuzzing:**
  ```bash
  ffuf -u https://example.com/?param=FUZZX -w dirs.txt:
FUZZ -w params.txt:FUZZX
  ```

* **403 Forbidden Bypass (Headers Injection):**
````
````

```bash
  ffuf -u https://example.com/admin -w 403-headers.txt
-H "FUZZ" -mc 200,301,302,307,401
  ```

* **403 Forbidden Bypass (`X-Forwarded-For` & `X-Origin
al-URL`):**
  ```bash
  ffuf -u https://example.com/FUZZ -w wordlist.txt -H
"X-Forwarded-For: 127.0.0.1" -H "X-Original-URL: /FUZZ"
  ```
```

- API Endpoint Discovery & Fuzzing
  - Kiterunner: [https://github.com/assetnote/kiterunner]
    A tool designed to perform API route discovery and brute-forcing using schema-aware wordlists (`.kite` files).

```
* **List Available Built-in Wordlists:**
  ```bash
  kr wordlist list
  ```

* **Scan API Routes (`/api/v1`):**
  ```bash
  kr scan https://api.example.com -A=apiroutes-260227:1
0000 -x 8 -j 15 -v info
  ```

* **Scan API Parameters (`/api/users?user_id=1`):**
  ```bash
  kr scan https://api.example.com -A=parameters-260227:
5000 -x 5 -j 10 -v info
  ```

* **Scan Directory Endpoints (`/uploads`):**
```
```

```bash
  kr scan https://api.example.com -A=directories-26022
7:8000 -x 6 -j 12 -v info
  ```

* **Batch Scan Multiple Targets from File:**
  ```bash
  kr scan targets.txt -A=directories-260227:8000 -x 6 -
j 12 -v info
  ```

* **Authenticated API Scan with Custom Header:**
  ```bash
  kr scan https://api.example.com -w wordlist.txt -H 'x
-access-token:123'
  ```
```

---

# Origin IP Discovery & Cloudflare/WAF Bypass Methodology

- **A comprehensive guide for discovering a website's real backend IP address (Origin IP) to bypass WAFs, Reverse Proxies, and CDNs (like Cloudflare or Akamai).**
  - ### What is Origin IP?

```
CDN / WAF IP:** The proxy server's IP address (e.g., Cloudflare: `104.x.x.x`) tha
t filters and routes traffic to the web application.
* **Origin IP:** The actual backend server IP address (`192.x.x.x`) hosting the w
eb application. Accessing the Origin IP directly bypasses WAF rules, rate limits,
and protection layers.
```

  - ### Tools & Commands from Image

```
originiphunter: [https://github.com/rix4uni/originiphunter]
An automated tool designed to discover origin IP addresses behind WAFs and CDNs b
```
```

```
y cross-referencing multiple OSINT and historical data providers.
```

```
* **Configuration:**
  ```bash
  nano ~/.config/originiphunter/config.yaml
  ```

* **Single Domain Scan:**
  ```bash
  echo "example.com" | originiphunter
  ```

* **Batch Scan Domains List:**
  ```bash
  cat domains.txt | originiphunter
  ```
```

- Additional Recommended Tools for Origin IP Discovery

```
CloudFail: [https://github.com/m0rt3za/CloudFail]
Unmasks Cloudflare IP addresses using misconfigured DNS re
cords, old DNS history, and Censys search queries.
```bash
python3 cloudfail.py --target example.com
```bash
```

```
Bypass-Firewalls-By-DNS-History: [https://github.com/vince
ntcox/bypass-firewalls-by-DNS-history]
Automates querying multiple historical DNS services (Secur
ityTrails, Crimeflare, DNSDumpster) to find old IP records
linked to the target.
```bash
```
```

````
bash bypass-firewalls-by-DNS-history.sh -d example.com
```
````

````
Censys Command Line Tool: [https://github.com/censys/censys-python]
Searches HTTPS SSL certificates to map certificates matching the domain to direct server IPs.
```bash
censys search "parsed.names: example.com"
```
````

````
FavFreak / Favicon Hashing: [https://github.com/devanshbatham/FavFreak]
Calculates the MD5/Murmur3 hash of the target website's `favicon.ico` file and queries Shodan/Censys for matching server hashes.
```bash
python3 favfreak.py -u https://example.com/favicon.ico
# Search Shodan using Murmur3 hash: http.favicon.hash:<hash_code>
````

- Manual Bypass Techniques & Ideas
  - MX & Mail Header Analysis :
    Send an email to a non-existent user on the target domain (`test12345@example.com`). Inspect the received "Non-Delivery Report" (bounce back email) headers for `Received: from` to find the internal/origin mail server IP.
  - Outbound SSRF / File Upload Callbacks:
    If the application allows importing images via URL, webhooks, or PDF generation, trigger an HTTP request back to a server you control (Burp Collaborator /

Interactsh) to capture the true backend IP in the request logs.

- Subdomain SSL Certificate Search (Shodan / Censys / FOFA):
  Query search engines for SSL certificates containing `CN=*.example.com` or `CN=example.com` to find servers directly exposing port 443 with the target's certificate.
- Historical DNS Queries:
  Check old A records using `SecurityTrails`, `ViewDNS.info`, or `DNSDumpster`. Many organizations configure Cloudflare *after* setting up the server, leaving the original A record logged in historical databases.
